Practical 3: Bank Customer Churn Prediction using Neural Network

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report,
f1_score
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

tf.keras.utils.set_random_seed(42)
tf.config.experimental.enable_op_determinism()
```

In [2]:
```
df = pd.read_csv("Churn_Modelling.csv")
df.head()
```

Out[2]:

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited
0	1	15634602	Hargrave	619	France	Female	42	2	0.00	1	1	1	101348.88	1
1	2	15647311	Hill	608	Spain	Female	41	1	83807.86	1	0	1	112542.58	0
2	3	15619304	Onio	502	France	Female	42	8	159660.80	3	1	0	113931.57	1
3	4	15701354	Boni	699	France	Female	39	1	0.00	2	0	0	93826.63	0
4	5	15737888	Mitchell	850	Spain	Female	43	2	125510.82	1	1	1	79084.10	0

In [3]:
```
df.shape
```

Out[3]:
```
(10000, 14)
```

In [4]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   RowNumber        10000 non-null  int64
 1   CustomerId       10000 non-null  int64
 2   Surname          10000 non-null  str
 3   CreditScore      10000 non-null  int64
 4   Geography        10000 non-null  str
 5   Gender           10000 non-null  str
 6   Age              10000 non-null  int64
 7   Tenure           10000 non-null  int64
 8   Balance          10000 non-null  float64
 9   NumOfProducts    10000 non-null  int64
 10  HasCrCard        10000 non-null  int64
 11  IsActiveMember   10000 non-null  int64
 12  EstimatedSalary  10000 non-null  float64
 13  Exited           10000 non-null  int64
dtypes: float64(2), int64(9), str(3)
memory usage: 1.1 MB
```

In [5]:
```
df.isnull().sum()
```

Out[5]:
```
RowNumber          0
CustomerId         0
Surname            0
CreditScore        0
Geography          0
Gender             0
Age                0
Tenure             0
Balance            0
NumOfProducts      0
HasCrCard          0
IsActiveMember     0
EstimatedSalary    0
Exited             0
dtype: int64
```

In [6]:
```
df.describe()
```

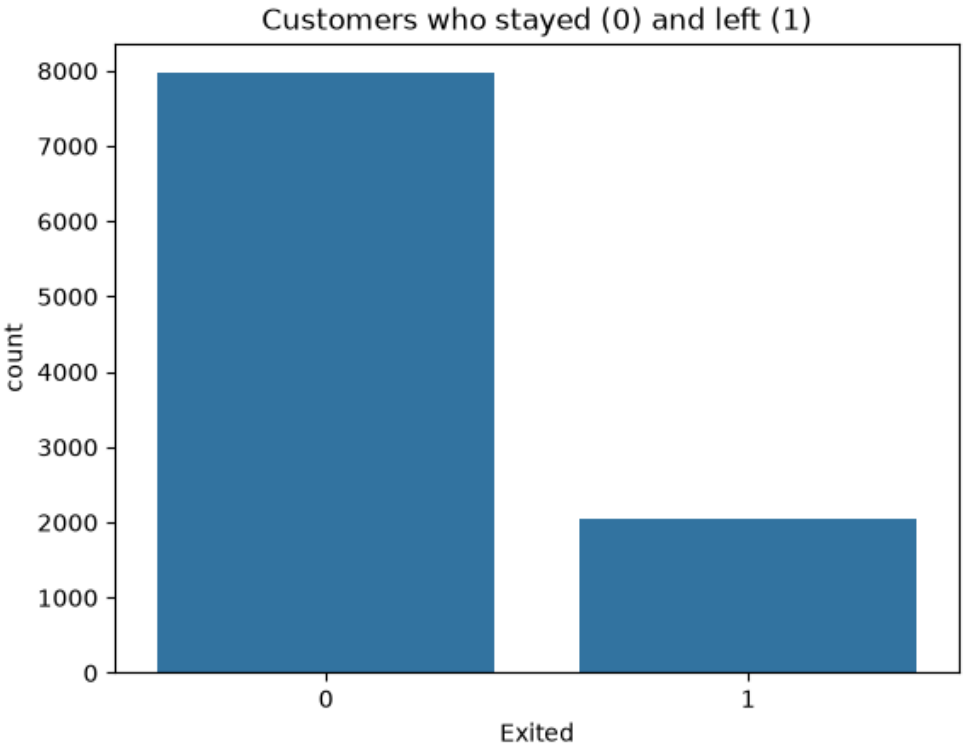
Out[6]:

	RowNumber	CustomerId	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited
count	10000.00000	1.000000e+04	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.00000	10000.000000	10000.000000	10000.000000
mean	5000.50000	1.569094e+07	650.528800	38.921800	5.012800	76485.889288	1.530200	0.70550	0.515100	100090.239881	0.203700
std	2886.89568	7.193619e+04	96.653299	10.487806	2.892174	62397.405202	0.581654	0.45584	0.499797	57510.492818	0.402769
min	1.00000	1.556570e+07	350.000000	18.000000	0.000000	0.000000	1.000000	0.00000	0.000000	11.580000	0.000000
25%	2500.75000	1.562853e+07	584.000000	32.000000	3.000000	0.000000	1.000000	0.00000	0.000000	51002.110000	0.000000
50%	5000.50000	1.569074e+07	652.000000	37.000000	5.000000	97198.540000	1.000000	1.00000	1.000000	100193.915000	0.000000
75%	7500.25000	1.575323e+07	718.000000	44.000000	7.000000	127644.240000	2.000000	1.00000	1.000000	149388.247500	0.000000
max	10000.00000	1.581569e+07	850.000000	92.000000	10.000000	250898.090000	4.000000	1.00000	1.000000	199992.480000	1.000000

In [7]:
```
df["Exited"].value_counts()
```

Out[7]:
```
Exited
0    7963
1    2037
Name: count, dtype: int64
```

In [8]:
```
sns.countplot(x="Exited", data=df)
plt.title("Customers who stayed (0) and left (1)")
plt.show()
```



In [9]:
```
df = df.drop(["RowNumber", "CustomerId", "Surname"], axis=1)
df = pd.get_dummies(df, columns=["Geography", "Gender"], drop_first=True, dtype=int)
df.head()
```

Out[9]:

	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited	Geography_Germany	Geography_Spain	Gender_Male
0	619	42	2	0.00	1	1	1	101348.88	1	0	0	0
1	608	41	1	83807.86	1	0	1	112542.58	0	0	1	0
2	502	42	8	159660.80	3	1	0	113931.57	1	0	0	0
3	699	39	1	0.00	2	0	0	93826.63	0	0	0	0
4	850	43	2	125510.82	1	1	1	79084.10	0	0	1	0

In [10]:
```
X = df.drop("Exited", axis=1)
y = df["Exited"]
print("Features:", X.shape)
print("Target:", y.shape)
```

```
Features: (10000, 11)
Target: (10000,)
```

In [11]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)
print("Training set:", X_train.shape, y_train.shape)
print("Test set:", X_test.shape, y_test.shape)
```

```
Training set: (8000, 11) (8000,)
Test set: (2000, 11) (2000,)
```

In [12]:
```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
pd.DataFrame(X_train, columns=X.columns).head()
```

Out[12]:

	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Geography_Germany	Geography_Spain	Gender_Male
0	1.058568	1.715086	0.684723	-1.226059	-0.910256	0.641042	-1.030206	1.042084	-0.578313	-0.577735	0.907507
1	0.913626	-0.659935	-0.696202	0.413288	-0.910256	0.641042	-1.030206	-0.623556	1.729169	-0.577735	0.907507
2	1.079274	-0.184931	-1.731895	0.601687	0.808830	0.641042	0.970680	0.308128	1.729169	-0.577735	-1.101919
3	-0.929207	-0.184931	-0.005739	-1.226059	0.808830	0.641042	-1.030206	-0.290199	-0.578313	-0.577735	0.907507
4	0.427035	0.955079	0.339492	0.548318	0.808830	-1.559960	0.970680	0.135042	1.729169	-0.577735	0.907507

In [13]:
```
tf.keras.utils.set_random_seed(42)
model = Sequential([
    Input(shape=(X_train.shape[1],)),
    Dense(6, activation="relu"),
    Dense(6, activation="relu"),
    Dense(1, activation="sigmoid")
])
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 6)	72
dense_1 (Dense)	(None, 6)	42
dense_2 (Dense)	(None, 1)	7

Total params: 121 (484.00 B)

Trainable params: 121 (484.00 B)

Non-trainable params: 0 (0.00 B)

In [14]:
```
history = model.fit(X_train, y_train, validation_split=0.2, epochs=50, batch_size=32,
verbose=0)
y_pred = (model.predict(X_test, verbose=0) > 0.5).astype(int).ravel()
acc1 = accuracy_score(y_test, y_pred)
cm1 = confusion_matrix(y_test, y_pred)
print("Accuracy:", round(acc1, 4))
print("Confusion Matrix:")
print(cm1)
```

```
Accuracy: 0.862
Confusion Matrix:
[[1532   61]
 [ 215  192]]
```

In [15]:
```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.88      0.96      0.92      1593
           1       0.76      0.47      0.58       407

    accuracy                           0.86      2000
   macro avg       0.82      0.72      0.75      2000
weighted avg       0.85      0.86      0.85      2000
```

Points of improvement

1. The network is very small (6 neurons per layer): use more neurons (32 and 16) so it can learn more patterns.
2. There is no regularisation: add Dropout layers to reduce overfitting.
3. Training for a fixed 50 epochs may stop too early or too late: train for up to 300 epochs with EarlyStopping and keep the best weights.
4. Only about 20% of customers leave, so the basic model misses more than half of them: give the customers who leave a higher class weight (1.5).

In [16]:
```
tf.keras.utils.set_random_seed(42)
model2 = Sequential([
    Input(shape=(X_train.shape[1],)),
    Dense(32, activation="relu"),
    Dropout(0.2),
    Dense(16, activation="relu"),
    Dropout(0.2),
    Dense(1, activation="sigmoid")
])
model2.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
model2.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 32)	384
dropout (Dropout)	(None, 32)	0
dense_4 (Dense)	(None, 16)	528
dropout_1 (Dropout)	(None, 16)	0
dense_5 (Dense)	(None, 1)	17

Total params: 929 (3.63 KB)
Trainable params: 929 (3.63 KB)
Non-trainable params: 0 (0.00 B)

In [17]:
```
early_stop = EarlyStopping(monitor="val_loss", patience=20, restore_best_weights=True)
history2 = model2.fit(X_train, y_train, validation_split=0.2, epochs=300, batch_size=32,
                      callbacks=[early_stop], class_weight={0: 1, 1: 1.5}, verbose=0)
print("Epochs trained:", len(history2.history["loss"]))
```

```
Epochs trained: 94
```

In [18]:
```
y_pred2 = (model2.predict(X_test, verbose=0) > 0.5).astype(int).ravel()
acc2 = accuracy_score(y_test, y_pred2)
cm2 = confusion_matrix(y_test, y_pred2)
print("Accuracy:", round(acc2, 4))
print("Confusion Matrix:")
print(cm2)
```

```
Accuracy: 0.8645
Confusion Matrix:
[[1532   61]
 [ 210  197]]
```

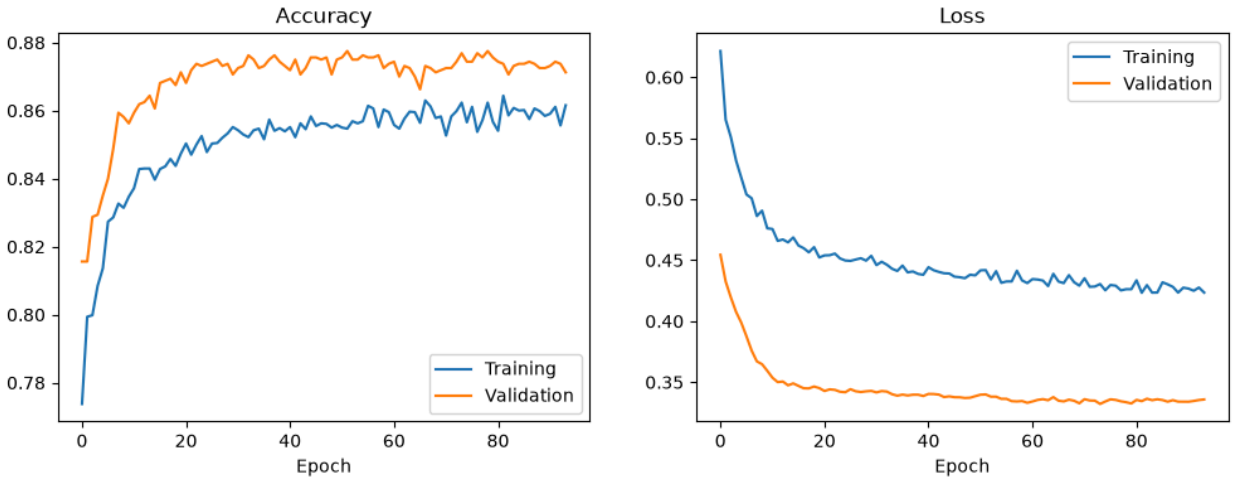
In [19]:
```
print(classification_report(y_test, y_pred2))
```

```
              precision    recall  f1-score   support

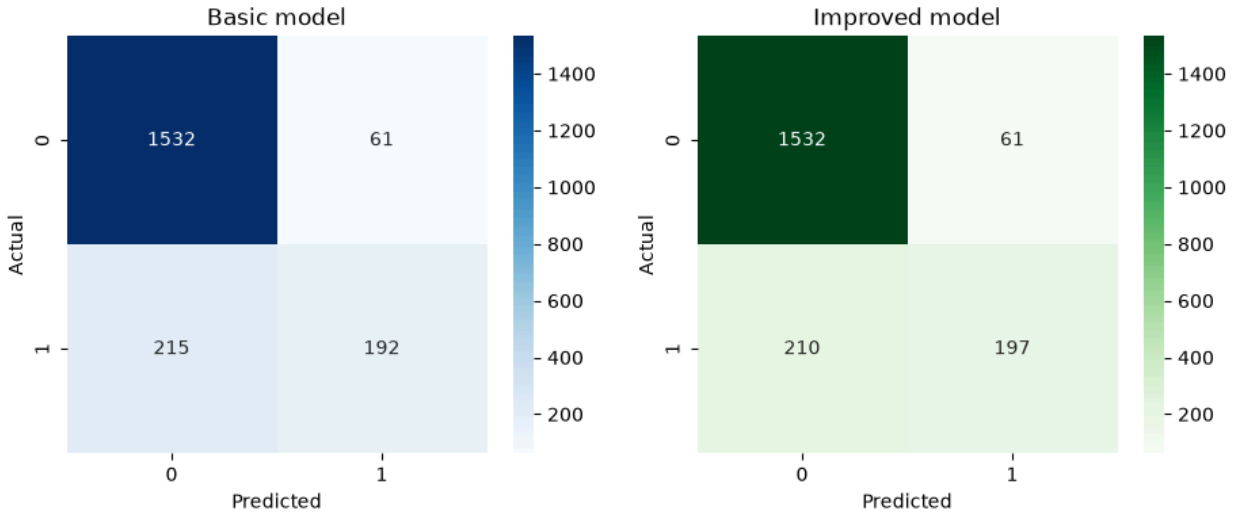
           0       0.88      0.96      0.92      1593
           1       0.76      0.48      0.59       407

    accuracy                           0.86      2000
   macro avg       0.82      0.72      0.76      2000
weighted avg       0.86      0.86      0.85      2000
```

In [20]:
```
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ax[0].plot(history2.history["accuracy"], label="Training")
ax[0].plot(history2.history["val_accuracy"], label="Validation")
ax[0].set_title("Accuracy")
ax[0].set_xlabel("Epoch")
ax[0].legend()
ax[1].plot(history2.history["loss"], label="Training")
ax[1].plot(history2.history["val_loss"], label="Validation")
ax[1].set_title("Loss")
ax[1].set_xlabel("Epoch")
ax[1].legend()
plt.show()
```



In [21]:
```
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
sns.heatmap(cm1, annot=True, fmt="d", cmap="Blues", ax=ax[0])
ax[0].set_title("Basic model")
sns.heatmap(cm2, annot=True, fmt="d", cmap="Greens", ax=ax[1])
ax[1].set_title("Improved model")
for a in ax:
    a.set_xlabel("Predicted")
    a.set_ylabel("Actual")
plt.show()
```



In [22]:
```
pd.DataFrame({
    "Model": ["Basic ANN (6-6-1)", "Improved ANN (32-16-1, Dropout, EarlyStopping, class weights)"],
    "Accuracy": [round(acc1, 4), round(acc2, 4)],
    "F1 score (left)": [round(f1_score(y_test, y_pred), 4), round(f1_score(y_test, y_pred2), 4)],
    "Customers who left, correctly found": [cm1[1, 1], cm2[1, 1]]
})
```

Out[22]:

	Model	Accuracy	F1 score (left)	Customers who left, correctly found
0	Basic ANN (6-6-1)	0.8620	0.5818	192
1	Improved ANN (32-16-1, Dropout, EarlyStopping,...	0.8645	0.5925	197